TAU - Advanced Topics in Programming, Semester B 2026

Assignment 2 Link to Assignment 1 Requirements

DRAFT PERIOD end date: June-7th
You should ask questions in the course forum. You'll get answers there also during the draft period. However, changes in the doc during the draft period won't be highlighted. After the draft period changes to this document will be communicated and highlighted.

Submission deadline: ~~June-21st~~ July-1st, 23:30.
Note: here are the Submission Instructions - please make sure to follow them!

Assignment 2 - Requirements and Guidelines

In this assignment the exact requirements for your API and file format will be formalized to create common API and usage.You will have to refactor your code to meet these exact requirements. (This is kinda "Refactoring Exercise").

In addition, you will have to implement "Component Testing" and "Integration Testing", both using GTest and GMock.

Updates:
9.6.26: Refactored IMap3D to accommodate MapConfig which include boundaries, offset and resolution. Aligned simulationRun and MissionManager to comply with the changes as well as refactored their types to accommodate a more accurate and decoupled design. Reworked the Maps Comparison utility and API to accommodate the changes.
Added cpp-yaml to ex_2 skeleton with an example build

12.6.26: changes in IMappingAlgorithm API - see more details under APIs.

15.6.26: Major change in ScanResultToVoxel which applies the voxels to the output map now. Changed MappingAlgorithm to take LidarConfigData and MissionConfigData in the constructor. Fixed MockGPS to have a field for the resolution, added to IMap3D a helper "isInBound".

20.6 - Minor changes:

- ILidar interface got a `config()` method
- `MissionConfigData` was added mission boundaries
- `SimulationCompositionData` was changed to enable nested simulation->mission configs

cont' next page...

Components:

- ***main*** - small, minimal, creates the SimulationManager and passes the relevant arguments to it.
- **SimulationManager** - Creates and manages all SimulationRun.
- ***ISimulationRun*** - manages the simulation, not relevant in the real world.
  - **SimulationRunImpl** - The actual single simulation working logic!
- **ISimulationRunFactory** - Factory for creating concrete ISimulationRun.
  - **SimulationRunFactory** - The actual concrete factory.
- ***IMissionControl*** - interface for the mission control.
  - **MissionControlImpl** - manages the mission, created in main (or in test), relevant both in simulation and in real world.
- ***IDroneControl*** - interface for drone control.
  - **DroneControlImpl** - manages the drone control, manages the drone movement decisions, lidar decisions, etc.
- **IMappingAlgorithm** - interface for the mapping algorithm, used solely by the drone control (the injection of mapping algorithm to drone control will be in the IDroneControl interface constructor).
  - **MappingAlgorithmImpl** - same class for simulation and real world.
- ***ILidar*** - interface for the Lidar, with following implementations:
  - LidarDriver - **not in scope**, a driver to the actual HW (here just for completeness).
  - **MockLidar** - used by the simulation and injected to the Drone Control.
- ***IGPS*** - interface for drone GPS, with following implementations:
  - GPSDriver - **not in scope** (here just for completeness)
  - **MockGPS** - used by the simulation and injected to the Drone Control
- ***IDroneMovement*** - interface for drone movement, with following implementations:
  - MovementDriver - **not in scope** (here just for completeness)
  - **MockMovement** - used by the simulation and injected to the Drone Control
- **IMap3D** - interface for the voxels map.
  - **Map3DImpl** - Used by a few components.
- **ScanResultToVoxels** - a utility class (provided by the course team). Takes the output of the LidarScan and applies it to the map.
- Helper Data Types (a partial list, full list in the reference stub implementation):
  - **LidarScanResult**
  - **MissionConfigData, DroneConfigData, SimulationConfigData, etc.**
- **MapsComparison** - a utility class for comparing simulation input map to generated output map. To be used by the Simulation or on its own as a standalone utility.

Note:
All components are **mandatory** (except those mentioned as “not in scope”) and you should implement them with the same name as presented above.

---

cont’ next page...

Map Input Files

Map input files will be .npy files, in binary format, as created by python visualization libraries like VoxelMap, Matplotlib, or PyVista, and as used by Minecraft.

Configuration Input Files - All files below are in YAML format

Drone

```
drone_config:
  dimensions_cm: 30      # sphere diameter drone can pass through
  max_rotate_deg: 45     # max rotation per command
  max_advance_cm: 50     # max advance distance per command
  max_elevate_cm: 40     # max elevate distance per command
```

Mission

```
mission_config:
  max_steps: 2400
  boundaries:
    x_boundary:
      min_cm: -500
      max_cm: -30
    y_boundary:
      min_cm: 30
      max_cm: 400
    height_boundary:
      min_cm: -30
      max_cm: 300
  gps_resolution_cm: 10         # GPS expected measurement precision
  # Optional: relative mapping resolution factor vs GPS
  # Integer. If missing, defaults to 1. If < 1, ignored with error log.
  output_mapping_resolution_factor: 2
```

Lidar

```
lidar_config:
  z_min_cm: 20                  # minimum operational distance
  z_max_cm: 120                 # maximum operational distance
  d_cm: 2.5                     # spacing between beam circles at z_min
  fov_circles: 5                # number of beam circles (0 to fov_c-1)
```

Single Simulation

```
simulation_config:
  map_filename: "maps/office_floor1.npy"
  map_resolution_cm: 10   # input file map voxel size (width/height/depth)
  initial_drone_position:
    x_cm: 250
    y_cm: 200
    height_cm: 150
  initial_angle_deg: 0    # 0=east, 90=south, 180=west, 270=north
```

```
map_axes_offset: // Turns (0,0,0)-> (x_offset,y_offset,height_offset)
      x_offset: 1000
      y_offset: 1000
      height_offset : 1500
```

Maps Comparison

```
comparison_config:
      original:
            map_res_cm: 10
            map_offset:
                  x_offset: 1000
                  y_offset: 1320
                  height_offset: 3200
            //need to ensure map_boundaries have size at most as the map.
            map_boundaries:
                  x_boundary:
                        min_cm: -500
                        max_cm: -30
                  y_boundary:
                        min_cm: 30
                        max_cm: 400
                  height_boundary:
                        min_cm: -30
                        max_cm: 300
      target:
            map_res_cm: 10
            map_offset:
                  x_offset: 100
                  y_offset: 132
                  height_offset: 32
            map_boundaries:
                  x_boundary:
                        min_cm: -500
                        max_cm: -30
                  y_boundary:
                        min_cm: 30
                        max_cm: 400
                  height_boundary:
                        min_cm: -30
                        max_cm: 300
```

Simulation Compositions

```
simulation_compositions:
  simulations:
    - simulation_config: "simulations/office_floor1_at_east.yaml"
      mission_configs:
        - "missions/office_floor1_east/mission_map_all_floor.yaml"
        - "missions/office_floor1_east/mission_map_east_part.yaml"
```

```
  - simulation_config: "simulations/office_floor1_at_west.yaml"
    mission_configs:
      - "missions/office_floor1_west/mission_map_all_floor.yaml"
      - "missions/office_floor1_west/mission_west_part.yaml"

  - simulation_config: "simulations/warehouse.yaml"
    mission_configs:
      - "missions/warehouse/mission.yaml"

drone_configs:
  - "drone_configs/drone_small.yaml"
  - "drone_configs/drone_large.yaml"

lidar_configs:
  - "lidar_configs/lidar_a.yaml"
  - "lidar_configs/lidar_b.yaml"
```

Note:
Above creates 20 single simulation runs, as the cartesian product of:
[mission_configs] * [drone_configs] * [lidar_configs]

---

cont' next page...

Output Files

Error Log

You should have an error log that will log all errors during the run.
The format of the error log and the log lines are yours!
All errors MUST be immediately logged to the error log file when they occur, and not deferred.

Map Output

Map output files will be .npy files, in binary format, similar to the input file, but might be in a different resolution.

Simulation Result Output File - YAML

```yaml
score_report:
  composition_file: "simulation_compositions.yaml"
  generated_at_utc: "2026-05-30T23:31:10Z"
  metric: "output_map_accuracy"
  score_range:
    min: 0
    max: 100
  error_score: -1

  summary:
    total_runs: 12
    scored_runs: 10
    error_runs: 2
    average_score: 87.4
    min_score: 61.2
    max_score: 98.9

  simulations:
    - simulation_config: "simulations/office_floor1_east.yaml"
      missions:
        - mission_config: "missions/office_floor1_east/mission_a.yaml"
          resolution_cm: 10   # the actual output resolution
          resolution_request_status: [IGNORED TOO SMALL/IGNORED/ACCEPTED]
          runs:
            - drone_config: "drone_configs/drone_small.yaml"
              lidar_config: "lidar_configs/lidar_a.yaml"
              status: "completed"
              steps: 1231
              score: 93.5 // scored by simulationManager

            - drone_config: "drone_configs/drone_small.yaml"
              lidar_config: "lidar_configs/lidar_b.yaml"
              status: "max_steps"
              steps: 2400
              score: 90.0
```

```
      - drone_config: "drone_configs/drone_large.yaml"
        lidar_config: "lidar_configs/lidar_a.yaml"
        status: "error"
        steps: 1320
        score: -1
        error_ref:
          code: "DRONE_HITS_OBSTACLE"

    - mission_config: "missions/office_floor1_east/mission_b.yaml"
      resolution_cm: 10   # the actual output resolution
      resolution_request_status: [IGNORED TOO SMALL/IGNORED/ACCEPTED]
      runs:
        - drone_config: "drone_configs/drone_small.yaml"
          lidar_config: "lidar_configs/lidar_a.yaml"
          status: "error"
          steps: 0
          score: -1
          error_ref:
            code: "MISSION_BOUNDARY_INVALID"
```

[...]

---

cont' next page...

APIs

Reference stub implementation to be presented by the course's staff (by Hiddai) in the upcoming recitation (3.6) and are available in the Github repository here.
Note a change in the API of ***IMappingAlgorithm*** done on **12/06** following notes raised in the exercise forum in moodle (more details on the change are in Hiddai's answer in that link).
Note a change in the API of **IMap3D**, **IMappingAlgorithm** and **ScanResultToVoxels** done on **15/6**. These came from the following moodle questions: 1, 2. More details in the answers to these questions.

Flow

Sequence diagrams to be presented by the course's staff (by Hiddai) in the upcoming recitation (3.6) and are available in the Github repository here.

MapsComparison utility

- Its API should meet the API as introduced in the reference stub implementation.
- No need for any interface, just this class as a standalone.
- This utility should also create an additional target (executable), with run instructions:

  ./maps_comparison <origin_map> <target_map> [comparison_config=<path>]

  - arguments:
    - *origin_map* and *target_map* will be the .npy file names (with or without path).
    - *Comparison_config is a path to a YAML file with the relevant configuration for executing the comparisons.* Providing a comparison_config file is optional, if not provided then assume both maps share the same offset, boundaries and resolution.
    - **Note:** support for comparing when the resolutions are **different** is an optional bonus feature!
  - The program will print to the standard output just the score number
    - As a floating point number between 0 and 100 - no additional text!
    - In case of an error: print to standard output the score -1 and to standard error an descriptive error message of your choice.

  The actual comparison algorithm is yours, but we will check that:
  1. Two identical maps return 100.
  2. Two very similar maps return a number close to 100, but not 100.
  3. Two very distinct maps return a number close to 0.
  4. Anything in between returns a reasonable result!

---

cont' next page...

Tests

Component Tests

You should create the following component tests, each should include as many tests as you feel are required to cover your code properly (happy path and error cases):

1. SimulationManager
2. SimulationRun - this component test should also test your mock implementations for the GPS and the DroneMovement.
3. MissionControl
4. DroneControl
5. MappingAlgorithm
6. MockLidar - even though this is a mock, since it's an important part of our simulation we want to have tests that verify its correct implementation.
7. MapsComparison

Each component testing should test that the implementation for the component alone is done right. Place the code of all component tests under: /tests/components/

Integration Tests

You should create at least two integration tests for the entire flow, each should include a few main flows of your program (focus on happy path):

1. Integration test of all components, including your real algorithm implementation.
2. Integration test of all components, but with a mock algorithm.

Place the code of the integration tests under: /tests/integration/

Important: how would we test your tests?

We will add intentional bugs into a specific component in your code and then run your tests. Our expectations are:

- Components tests that should not be affected by the change should not fail!
- The component test that should be affected by the change should detect at least 60% of the bugs that we introduce, by failing one or more tests.
- Your integration tests should detect at least 20% of the bugs that we introduce, by failing one or more tests.

Error Handling

You should follow the same instructions for error handling as in exercise 1.
Additionally: if an error occurs during the run and the simulator can continue to the next scenario, the failed scenario should get the score -1, indicating that it failed with an error and the simulation should continue to the next scenario.
In case an entire group of scenarios cannot run (e.g. error in map file that fails on scenarios that run on this map file), you can automatically fill the score -1 for all cases in that group.
In all cases, proper logs should be written to the error log, as in exercise 1. Note that all errors MUST be immediately logged to the error log file when they occur, and not deferred.

---

cont' next page...

Running the Program

Running all your tests should be done with the following command:

./drone_mapper_simulation [<simulation.yaml>] [<output_path>]

<simulation.yaml> should be replaced with the path and name of the simulation composition configuration yaml file.
You should support:

- missing the argument - fetch and use a file with the name “simulation.yaml” in the current working directory.
- argument contains only filename - look for this filename in the current working directory.
- relative path (starting without /) - look under the current working directory.
- absolution path (starting with a /) - retrieve the file from the provided path.

The program will create the following output in the provided output path, or if not provided, in the current working directory, overwriting existing files if needed:

- simulation_output.yaml – the scores of all the runs, organized hierarchically. You should decide how to manage the results in the file.
- a folder named “output_results” - with all maps and error logs organized under this folder. You should decide how to name the files, use nested folders if you wish.

You should document in your readme.txt the format that you have used for the simulation_output.yaml and for the output_results folder.

Running the Tests

Running all your tests should be done with the following command:

./drone_mapper_simulation_test

You should support the following test filters, to allow running certain specific tests:

./drone_mapper_simulation_test --gtest_filter=Integration.* (only integration tests)

And for all the specific components:

./drone_mapper_simulation_test --gtest_filter=SimulationManager.*

./drone_mapper_simulation_test --gtest_filter=SimulationRun.*

./drone_mapper_simulation_test --gtest_filter=MissionControl.*

./drone_mapper_simulation_test --gtest_filter=DroneControl.*

./drone_mapper_simulation_test --gtest_filter=MappingAlgorithm.*

./drone_mapper_simulation_test --gtest_filter=MockLidar.*

./drone_mapper_simulation_test --gtest_filter=MapsComparison.*

---

cont’ next page...

Bonuses

You may be entitled for bonus points for interesting implementation.
NOTE that implementing a smart algorithm will not be entitled for a bonus, as this would be part of your next assignments.
But, the following may be entitled for a bonus:

- adding visual simulation (do not make it mandatory to run the program with the visual simulation, you may base the visual simulation on the input and output files as an external utility, it can be written in C++ or in another language).
- supporting different resolution requests.

Note: do not ask for a bonus on a feature that was already submitted as a bonus in ex1!

BONUSES IMPORTANT NOTE: In order to get a bonus for any addition, you MUST add to your submission, inside the main directory, a *bonus.txt* file that will include a listed description of the additions for which you request for a bonus and how to check them, **when relevant add a test filter that specifically checks this bonus feature and provide it in this file together with the description of the bonus.**

---

cont' next page...

FAQ

Q:
Can we use AI when working on the assignments?

A:
You may (and even should) use GenAI tools, but remember that the responsibility for meeting the requirements and submitting working code is on you. Moreover, you are required to be able to explain any part of the code that you submit, this will be checked in short zoom meetings set with students separately, after your submission. You must read carefully and understand any piece of code that you integrate into your submission and be able to fully explain it.

Q:
Should we now have the perfect mapping algorithm?

A:
No. You should have a working algorithm with reasonable efficiency. But, in ex2 you are still not required to work too much on optimizing your algorithm. Still, if we have a small map that we feel should be mapped within a reasonable time (e.g. up to 10 seconds) and your algorithm takes much more (e.g. >2 minutes and still working), we will reduce points on that. If your algorithm tends to be very slow for all scenarios, the points reduction might be significant. And still, if it's only 3 times slower than another one, for ex2 we are still fine with that.

Q:
Can we deviate from the Interfaces and Data Types provided by the course staff?

A:
No, you should follow and use the exact Interfaces and Data Types provided.
The best way is to start from the skeleton project published for ex2.

Q:
Can we assume the same assumptions as in ex1?

A:
For any assumption that did not get a new definition, you should still assume it.
Examples:

- No need to handle battery recharging (we dropped it, it will not be in ex3 either).
- You may assume that the drone is a perfect sphere.
- You may assume that the lidar may point to any angle requested, with all other definitions as in ex1.

Q:
Can we still assume that there is a certain single supported resolution?

A:
Yes. But not exactly as in ex1. In this exercise the simulation can decide that it doesn't support the required resolution, report to the results that the requested resolution was ignored and use the default resolution that fits the mission.